INNOVATE MEMPHIS

StreetStat

Project Handoff Documentation

Prepared by Samarth Desai

Date July 14, 2026

Live site streetstat.org

Pedestrian crash & infrastructure context for Memphis — data, methodology, operations, and known open items.
This document is the complete, unabridged handoff: every caveat and open item is included by design.

Contents

StreetStat — Handoff Documentation

Written 2026-07-12 for the organization inheriting this project. Everything here was verified against the actual code and data on this date; where a fact could not be verified from the repo, it is marked as such.

---

1. What StreetStat is

StreetStat turns Tennessee's public crash records into a Memphis-specific accounting of pedestrian and non-motorist crashes **by road design and road ownership** — City of Memphis versus the Tennessee Department of Transportation (TDOT). It exists to give journalists and advocates a citable, verifiable alternative to "the pedestrian made a mistake" framing: every number on the site is a share, count, or distance computed from public data, reconciled against fixed totals (currently **1,337 crashes / 179 deaths**, Jan 1 2023 – Jul 11 2026), and stated descriptively — the site never claims a road *caused* a death. It ships as one self-contained web page (map, search, location reports, methodology) plus the reproducible Python pipeline that builds it.

2. Architecture

The four-part site (one page, four views)

VIEW	WHAT IT DOES
Home (`#/`)	Hero with computed stat cards + the findings dashboard (ownership split, charts, 25 deadliest corridors)
Explore (`#/explore`)	The citywide map. One analytic "lens" at a time (Road ownership / Sidewalk inventory / Crash density); crash dots always visible and clickable; signals and sidewalk segments clickable
Investigate (`#/investigate`)	The location microscope: address or coordinates in → full facts card (road, owner, sidewalk status, ±300 m crash count, time breakdown, nearest intersection/crossing) + hard-zoomed corridor view
Methodology (`#/methodology`)	Plain-language source → rule → limitations documentation of every pipeline stage

The page is **self-contained** (crash data and search index embedded, ~5.3 MB) so corridor and intersection search work even from a local file. Two Vercel serverless functions supplement it, neither needing any secret: `api/geocode.js` (address → coordinates via the US Census geocoder; needed because Census sends no CORS header) and `api/locate.js` (full-network street search — see the Addendum).

Which script generates what, and the build order

Scripts live in `scripts/`, numbered, and run in order. The full from-scratch order (verified by reading each script's declared inputs/outputs):

STEP	SCRIPT	PRODUCES
1	`01_download_crashes.py`	Raw crash pages (`data/raw/`), person-row CSV, deduplicated one-row-per-crash CSV
2	`02_download_roads.py`	`state_routes.geojson` , `memphis_boundary.geojson`
3	`03_spatial_join.py`	Memphis-filtered, first-pass-classified `shelby_crashes_classified.csv`
4	`05_download_streets.py`	`memphis_streets.geojson` — the full street network (~91 MB, gitignored; regenerate from the API)
5	`06_join_streets.py`	`shelby_crashes_named.csv` (nearest named street per crash), `deadliest_streets.csv`

STEP	SCRIPT	PRODUCES
6	`14_segment_jurisdiction.py`	Segment-level ownership (`shelby_crashes_named_seg.csv`, `ownership_segments.geojson`)
7	`17_classifier.py`	**The canonical classifier**: `shelby_crashes_final.csv`, `road_ownership_rulebook.geojson`, `ownership_segments_final.geojson`, `outputs/final_numbers.md`
8	`19` → `20` → `21`	TDOT pedestrian signals: raw → deduped crossings (one per intersection) → per-crash signal attributes + covered intersections
9	`22_osm_crossings_eval.py`	OSM crosswalk completeness report (read-only gate for step 10)
10	`23_union_poc.py`	Union Ave safe-crossing analysis: `union_safe_summary.json`, `union_poc.html`, report
11	*(manual step — no script)*	`data/processed/memphis_sidewalks_32136.geojson` from the sidewalk file-geodatabase (see §3)
12	`18_build_public_map.py`	`outputs/interactive_map/index.html` — the page shell, map, dashboard, methodology
13	`25_rebuild_junctions.py`	`intersection_nodes_all.geojson` (every junction citywide), `covered_corridors.json`, re-attributed `shelby_crashes_signals.csv`
14	`24_build_search.py`	`search_index.json` + injects search, Count-A, and the Investigate wiring **into** `index.html`

Scripts `04`, `07`–`12`, `15`, `16` are one-off statistics/audit steps or superseded prototypes — not needed for a rebuild.

Routine rebuild (data refresh, no methodology change): `01 → 03 → 06 → 14 → 17 → 21 → 23 → 18 → 25 → 27 → 24` (27 rebuilds the full-network search data so new streets/counts are searchable). Scripts 18 and 24 print a reconciliation (totals, 25-corridor match) on every run — **if it doesn't print OK, stop and investigate before deploying.** Script 18's check is internal consistency against the *current* data; some older scripts (21, 25) additionally print comparisons against the *previous* anchors as information — expect those to read "False/CHECK" on a legitimate refresh. The sanity anchors in `CLAUDE.md` shift as the state's rolling window advances; update them after each verified refresh rather than assuming.

View locally: `.\.venv\Scripts\python.exe -m http.server 8000 --directory outputs\interactive_map` then open `http://localhost:8000/index.html`. (Address search needs the deployed `/api` functions.)

3. Data sources — origin, refresh, caveats

Crashes

ORIGIN. TDOT SAFETY MapServer, layer 8 (`tnmap.tn.gov/arcgis/rest/services/SAFETY/MapForDashboards/MapServer/8/query`) — public, no key. Full reference: `outputs/data_source.md`

REFRESH. Re-run `scripts/01_download_crashes.py` (it re-downloads when the upstream count changes), then the routine rebuild chain

CAVEATS. One row per **person**, deduped to crashes; pedalcyclists excluded by design. **Rolling ~3-year window** — old records fall out as new ones arrive. **Freshness measured:** newest upstream record was 1 day old when probed 2026-07-12 (typically ~1–3 days), but recency ≠ completeness — police reports are finalized with a lag, so the last several weeks always undercount. **Upstream records can also be REMOVED outside the rolling window** (observed in the first live refresh, 2026-07-13: one fatal record deleted at the source between the May and July pulls — not renumbered, not reclassified). Deletions are invisible to date-floor incremental pulls; only a full pull catches them.

Roads / boundary / street network

ORIGIN. City of Memphis Public Works GIS (ArcGIS REST)

REFRESH. `02` (routes/boundary), `05` (street network)

CAVEATS. Network is the city's centerline file; `LANES` / `SPDLIMIT` attributes are the city's, taken as-is

Sidewalk inventory

ORIGIN. City of Memphis (Engineering) — received June 2026 as `Memphis_Sidewalks_DMC (1).zip` → a `Memphis_Sidewalks_V2` file-geodatabase (zip dated April 2026); 46,875 lines with `STREET_NAME` and `WIDTH`. **Redistribution permission granted and recorded July 2026** — the reprojected working file is now committed at `data/processed/memphis_sidewalks_32136.geojson` (the source zip stays out of the repo)

REFRESH. No refresh endpoint — a new delivery would come from the city. To regenerate the working file: read the GDB layer with geopandas and write `data/processed/memphis_sidewalks_32136.geojson` in EPSG:32136 *(this conversion was a one-off manual step; there is no numbered script for it)*

CAVEATS. **Vintage unknown** — the GDB carries no collection-date metadata (internal file timestamps are Jan 2025, which reflects export, not survey date); this caveat stands regardless of permission status. This is why the site only ever says "in city inventory" / "none found in city inventory (absence may reflect incomplete records)" — never a flat "no sidewalk."

Pedestrian signals

ORIGIN. TDOT “ADA Asset Data” FeatureServer (geodata.tn.gov Hub item `69511fa73a584e2bb37acfa85b177fa5`, layer 1)

REFRESH. Re-run `19 → 20 → 21`, then `25 → 24`

CAVEATS. An asset inventory: it records where TDOT has inventoried signals, mostly along state routes. Off covered corridors the site says “not yet analyzed” — absence of inventory is not “no signal”

OSM crosswalks

ORIGIN. OpenStreetMap via Overpass (ODbL) — `22` acquires and evaluates

REFRESH. Re-run `22`, review the report, then `23`

CAVEATS. **Union Avenue only** so far. OSM completeness varies block to block; `outputs/osm_crossings_eval.md` recommends an imagery spot-check before extending citywide. ODbL attribution required

Geocoding

ORIGIN. US Census Bureau geocoder via `api/geocode.js`

REFRESH. none needed

CAVEATS. Free, no key; occasionally misses newer addresses

4. Known open items (honest and complete)

1. **AI drafting layer: removed.** An AI drafting layer ("Report a New Incident") was prototyped during development and **removed before launch**; the code remains retrievable from git history (removed 2026-07-13). If IM ever builds an AI feature, it must use IM's own API key and never commit credentials. The deterministic facts API the prototype was built on (`window.CountA.facts`) is a live, tested part of the product — the Investigate view runs on it.
2. **Safe-crossing / longest-gap statistics are preliminary.** The Union Ave numbers (24 safe crossings, 22% of crossing-relevant crashes >250 ft, 2,921 ft longest gap) are a **proof of concept on one corridor**, pending imagery ground-truthing of the OSM crosswalk layer (`outputs/osm_crossings_eval.md` describes the recommended check). They are labeled "preliminary" in the README, the Union report, and on the site's Union card. Do not extend citywide before ground-truthing.
3. **Live-update pipeline: built; the daily schedule activates by uncommenting the `cron` block** in `.github/workflows/data-refresh.yml` (manual `workflow_dispatch` runs are available now). No secrets or setup required — see §5.
4. **Sidewalk data vintage unknown.** (Redistribution permission was granted and recorded July 2026 — that half of the item is closed; the unknown survey vintage remains.) See §3.
5. **Name collision:** "streetStat" is also the name of a Massachusetts pharmaceutical consultancy (streetStat LLC). Known and accepted by the project owner; revisit only if the project seeks trademark or wide distribution. *(Not verifiable from the repo — recorded from the project owner.)*
6. **Superseded figure in the living stats document.** `novel_statistics.docx` contains an early "~50/50 signalized/unsignalized" intersection split; a dated correction appended 2026-07-12 supersedes it with the current verified **39.9% signalized / 60.1% unsignalized** (298 covered at-intersection crashes). Cite the correction, not the original.
7. **Nothing else is TODO-flagged.** A code sweep found no other TODO/FIXME markers; the only "beta" wording is the AI note above.

5. Live updates — the automated refresh (GitHub Actions)

Data refreshes are **automated**: a scheduled GitHub Action (`.github/workflows/data-refresh.yml`) runs the refresh engine `scripts/28_auto_refresh.py`. The manual procedure further below is retained as the fallback and is exactly what the automation executes.

- **What runs when.** Daily at 08:47 UTC (~3:47 AM Memphis): an *incremental* check — a cheap date-floored probe (last snapshot max minus a 30-day overlap); if nothing changed, the job exits without committing. On the **1st of each month** the same workflow runs a *full* unconditional re-pull — required, not optional, because the source both **backfills** old dates and **deletes records** (proven 2026-07-13), and neither is visible to a date-floored pull. When the probe does detect changes, the engine re-pulls the whole window anyway (it is a single API page at this data size — same result as a merge, zero merge-drift risk).
- **The safety gates** (all inside `28_auto_refresh.py`; the job physically cannot publish without passing them): pull plausibility (API count must succeed, be ≥90% of local rows, and match the rows actually downloaded) → every pipeline step must exit 0 → internal reconciliation computed from the fresh outputs (surface + limited-access partitions the total; the search and locate indexes sum back to it) → sanity bounds vs the previous snapshot (new total ≥ old − 5 and ≤ old + 75; max CollisionDate never goes backward) → and a no-change short-circuit (zero added/removed/modified records → clean exit, no commit, no deploy). Auto-commits are labeled `auto: data refresh through YYYY-MM-DD — N new, M removed`.
- **Where failures surface:** any abort or error **opens a GitHub Issue** on the repo (visible to all collaborators — that is the alert channel). The live site is never touched on failure.
- **Pause it:** repo → Actions → "Data refresh" → ⋯ menu → *Disable workflow*. Re-enable the same way. **Run it manually:** same page → *Run workflow* (choose mode, optionally dry-run — a dry run executes everything including the gates but never commits).
- **No secrets, no setup.** The sidewalk inventory is committed to the repository (redistribution permission granted July 2026), so a fresh checkout has everything except the 91 MB street network, which the job restores from the Actions cache or re-downloads from the public city API (script 05). The workflow uses only the built-in `GITHUB_TOKEN`. **Enabling the schedule** = uncomment the `cron` block at the top of `.github/workflows/data-refresh.yml` (manual *Run workflow* is available regardless).
- **Cost:** the repository is public, so GitHub Actions minutes are **free and unlimited**; the run takes ~8–12 minutes (≈250–360 min/month — comfortably inside even the private-repo free tier of 2,000 min/month, if the repo ever goes private).

Background: the endpoint, and the manual fallback

Verified against the endpoint (details and tested queries in `outputs/data_source.md`): the crash layer is a standard ArcGIS REST `/query` endpoint that supports **date filtering** (`CollisionDate` is a true date field), **pagination** (`resultOffset` / `resultRecordCount=2000`), and **cheap count-only probes**. The design the automation implements:

- **Incremental pull (daily in the automation).** Query with the existing `where` clause **plus a CollisionDate floor**: `AND CollisionDate ⩾ DATE '<local_dmax minus 30 days>'` (the 30-day overlap catches late-arriving reports for recent dates). Page through, dedupe person-rows to crashes, and merge into the local crash file **keyed on** **`MstrRecNbrTxt`** (replace matching ids, append new ones — never blind-append).
- **Monthly full refresh.** Re-run the unfiltered pull (script `01` already does this with a count-change check). This is **required, not optional**, for two things incremental pulls cannot see: **backfills/corrections** to older records, and the **trailing edge of the rolling ~3-year window** (old crashes silently drop out upstream; totals are only correct after a full pull).
- **After either pull:** run the routine rebuild chain (§2), require the printed reconciliation to pass (totals will legitimately drift as the window advances — the check is that surface + limited-access sums match the new dedup total, and the 25-corridor table matches the index), update the sanity anchors in `CLAUDE.md`, and redeploy. If reconciliation fails, do not deploy.
- **Scheduling:** GitHub Actions cron (or any weekly scheduler) is sufficient; total runtime is minutes. Keep the raw page dumps it writes out of git (`data/raw/` is already the convention).
- **First live run of this procedure (2026-07-13), for calibration:** full pull moved the data from 1,294/175 (through 2026-05-26) to **1,337/179 (through 2026-07-11)**: 44 new crashes (5 fatal) after the old cutoff, **0 backfilled** records before it, and **1 record REMOVED upstream** (a fatal; verified absent from the source even unfiltered, and not renumbered — see the Crashes caveat above). The zero-backfill result doesn't invalidate the 30-day overlap (reporting lag guarantees late arrivals eventually); the deletion is the concrete proof that the monthly FULL pull is mandatory.
- **Display:** the "Data current through …" labels on the site are computed from the data at build time, so they update automatically.

6. Operating costs and monthly maintenance

Costs (as configured today): - **Hosting: $0.** A static page + two small serverless functions fit Vercel's free (Hobby) tier. No database, no paid APIs, and no secrets of any kind in the current build. - **Data: $0.** All sources are public endpoints without keys. - **Domain:** currently the free `*.vercel.app` URL; a custom domain would be the only recurring cost (~$10–20/yr).

Monthly maintenance (~1 hour; data refreshes themselves are automated, §5): 1. Skim the Action's recent runs and any refresh-failure Issues; confirm the latest auto-commit's reconciliation summary looks sane. (Manual fallback: run `01` + the routine rebuild chain.) 2. Update the sanity anchors in `CLAUDE.md` to the new totals; skim the findings page for anything that reads oddly against the new numbers. 3. Redeploy (push the rebuilt `outputs/interactive_map/` to Vercel) and spot-check: one corridor search, one intersection, one Investigate lookup, one address (exercises `/api/geocode`). 4. Once a quarter: re-pull signals (`19` – `21` → `25` → `24`) and re-check that the source endpoints haven't changed schema (script `01` will fail loudly if the crash API changes).

7. How to cite StreetStat

> StreetStat — pedestrian crash & infrastructure context for Memphis. Built by Samarth Desai, in support of pedestrian-safety advocacy with Street Fair Memphis and Innovate Memphis. https://streetstat.org/ (accessed *date*). Crash data: Tennessee SAFETY MapServer (TDOT), rolling window from Jan 1 2023 *(cite the "data current through" date shown on the site at access time).* Roads, boundary, and sidewalk inventory: City of Memphis Public Works. Pedestrian signals: TDOT ADA Asset Data. Crosswalks: © OpenStreetMap contributors (ODbL).

When citing a specific number, prefer the wording the site itself uses (shares of *surface-street* crashes, deaths *on roads with* 4+ lanes, etc.) — the qualifiers are part of the finding. For the methodology behind any figure, cite the site's Methodology page or `novel_statistics.docx` (including its 2026-07-12 correction section).

8. Two ways for Innovate Memphis to use StreetStat

Both paths below are open to you, and they are not mutually exclusive.

Which path do I want? Improving StreetStat itself → **Path A**. Building something of IM's own on top of it → **Path B**. Doing both at once is fine.

Path A — working on the platform directly

IM team members will be added as collaborators on the repository, which gives you everything: the ability to edit, maintain, and improve StreetStat itself — the map, the analysis, the data refreshes, all of it.

> **The one thing to internalize before your first push: changes pushed to this repository deploy automatically to the live public site.** There is no separate "publish" step — a push to `main` *is* publishing, and the deploy typically goes live in under a minute.

The safe-change workflow, every time:

1. **Pull the latest** (`git pull`) so you start from exactly what is live.
2. **Edit locally.**
3. **Rebuild** using the documented script order (§2). For presentation changes, `scripts/18_build_public_map.py` then `scripts/24_build_search.py` is enough; after a data refresh, run the full routine chain.
4. **Read the reconciliation the build prints** — it checks the page's numbers against the current data totals. **If it prints FAIL, STOP.** Do not push; something is inconsistent, and pushing would publish it.
5. **Test on localhost:** `.\.venv\Scripts\python.exe -m http.server 8000 --directory outputs\interactive_map` then open `http://localhost:8000/index.html` and click through what you changed.
6. **Commit and push.**

A suggestion (not a rule): work through **pull requests with one teammate's review before merge**. It costs a few minutes and means no single push can break the live public site.

Never commit, under any workflow: - **API keys or secrets of any kind.** The current build needs none — if a future feature ever does (e.g. an AI service), its credentials belong in Vercel's environment settings (or a gitignored local file for dev), never in the repository. - **The gitignored large data files** — `data/raw/memphis_streets.geojson` (~91 MB street network; script 05 regenerates it), the sidewalk delivery (`Memphis_Sidewalks_DMC*.zip` and `data/processed/memphis_sidewalks_32136.geojson` — also pending redistribution permission, §3), and the raw API page dumps (`data/raw/*_page_*.json`).

The `.gitignore` already blocks all of these — never override it with `git add -f`.

Path B — copying the platform for IM's own use

For **Data Midsouth** or any other IM endeavor: **fork the repository** on GitHub (or clone / download a copy). That gives IM a complete, independent copy of all code and data — IM's to modify, host, and deploy anywhere, for any purpose, **no permission needed**: the code is MIT-licensed (source data remains under its providers' terms, §3; note the sidewalk layer's pending redistribution permission before republishing that file).

> Equally important, in the other direction: **changes to a copy never affect the live StreetStat site** — experiment freely. And a copy does **not** receive future StreetStat updates automatically: when you want the latest, re-sync from the repository (GitHub's "Sync fork" button, or `git pull` from the original remote) — entirely on your schedule.

Code is MIT-licensed. Source data remains under its providers' terms. The pipeline's own rule, worth keeping: compute every statistic from the data files — never hardcode a number into an output.

Addendum (2026-07-12, same night): full-network search

The search overhaul added after this document was first written:

- **`scripts/27_build_locate_index.py`** (runs after 25, before 24) preprocesses the full street network into a compact lookup — `outputs/interactive_map/api/locate_data.json` (~2.9 MB: all 16,719 named Memphis streets with bbox/length/owner/crash-count, the 25,533-junction index, and a 97-key state-route alias table derived from the data).
- **`api/locate.js`** — a third Vercel serverless function: `GET /api/locate?q=...` answers street and intersection queries over the full network with forgiving matching (case/suffix/ directional-blind, “and”/”&”/”@”, 1–2-char typos, aliases). Bundles the JSON via `require`; measured cold start ~0.3–0.4 s, warm queries under ~55 ms. Test it locally with `node scripts\locate_dev_server.js`.
- The page’s own matcher gained the same forgiveness for the embedded index, and calls `/api/locate` only when the embedded index can’t resolve a query. A street outside the 529 crash corridors returns an honest minimal card (0 incidents recorded, owner from the rulebook, “not analyzed” for sidewalk/stretch fields — never fabricated analysis).

Needs the deployed server (won’t work on `file://`): address search (`/api/geocode`) and full-network street lookup (`/api/locate`). Everything else — map, lenses, feature popups, corridor/intersection search including casual/typo/alias forms, coordinate lookups, Investigate coordinates mode — is embedded and works offline. The page says so honestly when offline.